

STRIDE Analyzer

Documentação Técnica Completa

Arquitetura · Roteiro de Execução · Referência de Comandos

Tech Challenger — Fase 5

FIAP · Postech

Pacote: stride-analyzer v1.0.0

Python >= 3.10 · Docker Compose · Twelve-Factor App

Desenvolvido por Bruno José e Silva - brunojose1977@yahoo.com.br - julho 2026

Julho/2026

1. Sumário

1. Sumário
2. Visão geral do projeto
3. Arquitetura da solução
4. Estrutura de diretórios
5. Módulos do pacote Python
6. Fluxo de dados (pipeline)
7. Configuração e variáveis de ambiente
8. Arquitetura Docker
9. Princípios Twelve-Factor
10. Roteiro passo a passo — execução local
11. Roteiro passo a passo — execução com Docker
12. Referência de comandos CLI
13. Testes e qualidade
14. Geração de relatórios
15. Solução de problemas

Desenvolvido por Bruno José e Silva - bruno jose1977@yahoo.com.br - julho 2026

Desenvolvido por Bruno José e Silva - bruno jose1977@yahoo.com.br - julho 2026

2. Visão geral do projeto

O STRIDE Analyzer é um projeto Python de produção derivado do notebook Colab prototipo_carga_de_diagramas_do_dataset_do_Kaggle_e_análise_STRIDE_v6. Sua função é automatizar a análise de ameaças STRIDE sobre diagramas de arquitetura de software.

O pipeline realiza quatro etapas principais:

- Download — baixa diagramas de arquitetura do dataset Kaggle carlosrian/software-architecture-dataset.
- Filtragem — avalia nitidez (Laplaciano/OpenCV), alinhamento (Hough) e classificação zero-shot (CLIP) para selecionar diagramas válidos.
- Análise STRIDE — envia cada diagrama aprovado ao modelo GPT-4o Vision (OpenAI) para modelagem de ameaças.
- Relatórios — consolida o resultado em JSON, PDF e DOCX na pasta relatorio/.

2.1 Objetivos acadêmicos e técnicos

O projeto demonstra a conversão de um protótipo exploratório (notebook) em um pacote modular instalável via pip, com CLI, testes, configuração por variáveis de ambiente e orquestração Docker alinhada aos princípios Twelve-Factor App.

3. Arquitetura da solução

A aplicação é uma CLI stateless organizada em camadas: entrada (CLI), configuração, domínio (dataset, análise de imagem, STRIDE) e saída (relatórios). Não há banco de dados; o estado persiste apenas em volumes/pastas locais.

Tipo	Componente	Responsabilidade
Camada	Interface (CLI)	argparse; entry point stride-analyzer
Camada	Configuração	Settings imutável via env (.env / Twelve-Factor III)
Camada	Domínio	dataset, filter, image_analysis, stride
Camada	Saída	reports (JSON / PDF / DOCX)
Externo	Kaggle API	Download do dataset de diagramas
Externo	Hugging Face / CLIP	Classificação zero-shot de imagens
Externo	OpenAI GPT-4o Vision	Análise de ameaças STRIDE

Tabela 1 — Componentes lógicos da arquitetura

3.1 Diagrama lógico do pipeline

CLI (stride-analyzer) → Settings.from_env() → setup (pastas) → download (Kaggle) → filter (OpenCV + CLIP) → diagramas_selecionados/ → analyze (GPT-4o Vision) → relatorio/ (JSON + PDF + DOCX)

O comando run-all encadeia download → filter → analyze. No Docker Compose com profile modular, cada etapa é um contêiner independente com depends_on.

4. Estrutura de diretórios

```
bruno jose1977-fiap-tech-challenger-fase5/  
├── src/stride_analyzer/ # Pacote da aplicação  
├── tests/ # Suite pytest  
├── docker/entrypoint.sh # Entrypoint do contêiner  
└── docs/ # Documentação e evidências
```

```

├── dataset/ # Dataset Kaggle (runtime)
├── diagramas_selecionados/ # PNGs filtrados (runtime)
├── relatorio/ # Relatórios gerados (runtime)
├── minha-seleção-diagramas/ # Diagramas curados manualmente
├── Dockerfile / docker-compose.yml
├── pyproject.toml / .env.example
└── README.md

```

As pastas `dataset/`, `diagramas_selecionados/` e `relatorio/` são criadas pelo comando `setup` (ou pelo entrypoint Docker). O conteúdo gerado em runtime é ignorado pelo Git.

5. Módulos do pacote Python

Módulo	Responsabilidade
<code>cli.py</code>	Entrada do processo: parser argparse, logging, comandos <code>setup/download/filter/analyze/run-all</code>
<code>config.py</code>	Classe <code>Settings</code> ; carrega <code>.env</code> ; resolve <code>PROJECT_ROOT</code> ; valida credenciais Kaggle/OpenAI
<code>paths.py</code>	Helpers de filesystem: verificar pastas, listar PNGs, checar dataset local
<code>dataset.py</code>	Download via Kaggle CLI, extração de ZIP, remoção de XML
<code>image_analysis.py</code>	Nitidez (Laplaciano), alinhamento (Hough), CLIP zero-shot, <code>avaliar_imagem</code>
<code>filter.py</code>	Percorre o dataset, aplica critérios e copia até <code>MAX_IMAGENS</code> para <code>diagramas_selecionados/</code>
<code>stride.py</code>	Codifica imagem em base64, chama GPT-4o Vision com <code>PROMPT_SEGURANCA</code> , batch
<code>reports.py</code>	Exporta <code>analise_stride_*.json</code> , <code>relatorio_stride_*.pdf</code> e <code>*.docx</code>

Tabela 2 — Mapa de módulos em `src/stride_analyzer/`

5.1 Critérios de filtragem de imagem

- Nitidez — variância do Laplaciano $\geq$ `SHARPNESS_THRESHOLD` (padrão 150.0).
- Alinhamento — detecção de linhas via Transformada de Hough.
- CLIP — classificação zero-shot contra rótulos de diagrama de arquitetura (`ARCHITECTURE_LABELS`).

A função `avaliar_imagem` exige que todos os critérios sejam aprovados. O comando `analyze` processa apenas arquivos `.png` no nível raiz de `diagramas_selecionados/`.

6. Fluxo de dados (pipeline)

6.1 Etapa setup

Cria as pastas de trabalho e imprime o status (OK/AUSENTE) com contagem de arquivos e imagens. Não depende de credenciais externas.

6.2 Etapa download

Se já existirem imagens em `dataset/`, o download é ignorado (idempotência). Caso contrário: valida credenciais Kaggle, baixa o dataset com a CLI Kaggle (`--unzip`) e remove arquivos `.xml`.

6.3 Etapa filter

Garante o dataset, instancia o classificador CLIP (salvo com `--skip-clip`), percorre imagens com `rglob` e copia as aprovadas até o limite `MAX_IMAGENS`.

6.4 Etapa analyze

Lista PNGs selecionados; para cada um chama a API OpenAI Vision com o prompt STRIDE; agrega os resultados e grava os três formatos de relatório com timestamp YYYYMMDD_HHMMSS. Retorna código de saída 1 se não houver PNGs.

7. Configuração e variáveis de ambiente

Variável	Obrigatória	Padrão	Descrição
KAGGLE_USERNAME	Sim (download)	—	Usuário da API Kaggle
KAGGLE_KEY	Sim (download)	—	Token da API Kaggle
OPENAI_API_KEY	Sim (analyze)	—	Chave da API OpenAI
PROJECT_ROOT	Não	auto	Raiz do projeto (em Docker: /app)
MAX_IMAGENS	Não	10	Máximo de diagramas filtrados
SHARPNESS_THRESHOLD	Não	150.0	Limiar de nitidez (Laplaciano)
KAGGLE_DATASET	Não	carlosrian/...	Slug do dataset Kaggle
OPENAI_MODEL	Não	gpt-4o	Modelo Vision OpenAI

Tabela 3 — Variáveis de ambiente (.env.example)

Secrets nunca devem ser commitados. O arquivo .env está no .gitignore. Em Docker Compose, env_file: .env é opcional (required: false).

8. Arquitetura Docker

A imagem é baseada em python:3.11-slim-bookworm, com bibliotecas sistêmicas para OpenCV (libgl1, libglib2.0-0, libgomp1). O entrypoint docker/entrypoint.sh garante as pastas de volume e delega para stride-analyzer. CMD padrão: run-all.

Serviço	Profile	Responsabilidade
setup	modular	Cria pastas e valida volumes
download	modular	Baixa dataset do Kaggle
filter	modular	Filtra diagramas (limite 4G RAM)
analyze	modular	Análise STRIDE + relatórios
pipeline-modular	modular	Alpine: confirma sucesso da cadeia
pipeline	monolith	run-all em um único contêiner

Tabela 4 — Serviços do docker-compose.yml

8.1 Volumes

Mount	Conteúdo
./dataset → /app/dataset	Imagens do Kaggle
./diagramas_selecionados → /app/diagramas_selecionados	Diagramas filtrados
./relatorio → /app/relatorio	JSON, PDF e DOCX
hf-cache (nomeado stride-hf-cache)	Cache do modelo CLIP (Hugging Face)

Tabela 5 — Volumes compartilhados

9. Princípios Twelve-Factor

Fator	Implementação
I. Codebase	Repositório único; código em src/stride_analyzer/
II. Dependencies	pyproject.toml com dependências explícitas
III. Config	Variáveis de ambiente via .env (nunca commitadas)
IV. Backing services	Kaggle e OpenAI como recursos anexáveis
V. Build, release, run	Build da imagem Docker separado da execução
VI. Processes	CLI stateless (stride-analyzer)
VII–VIII. Concurrency	Processos/contêineres independentes por comando
IX. Disposability	Comandos iniciam e encerram rapidamente
X. Dev/prod parity	Mesma CLI e config em local e Docker
XI. Logs	Saída em stdout
XII. Admin processes	Comandos setup, download, filter, analyze

Tabela 6 — Mapeamento Twelve-Factor

Desenvolvido por Bruno José e Silva - bruno jose1977@yahoo.com.br - julho 2026

Desenvolvido por Bruno José e Silva - bruno jose1977@yahoo.com.br - julho 2026

10. Roteiro passo a passo — execução local

10.1 Pré-requisitos

- Python 3.10 ou superior instalado e no PATH
- Conta Kaggle com token de API (Account → API → Create New Token)
- Chave da API OpenAI com acesso a GPT-4o Vision
- Espaço em disco suficiente para o dataset e cache do CLIP (recomendado ≥ 4 GB livres)

Passo 1 — Acessar o projeto

```
cd caminho\para\brunojose1977-fiap-tech-challenger-fase5
```

Passo 2 — Criar e ativar ambiente virtual

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

Passo 3 — Instalar dependências

```
pip install --upgrade pip  
pip install -e ".[dev]"
```

Passo 4 — Configurar .env

```
copy .env.example .env  
# Edite KAGGLE_USERNAME, KAGGLE_KEY e OPENAI_API_KEY
```

Passo 5 — Verificar pastas

```
stride-analyzer setup
```

Passo 6 — Baixar dataset

```
stride-analyzer download
```

Passo 7 — Filtrar diagramas

```
stride-analyzer filter  
# Opcional mais rápido: stride-analyzer filter --skip-clip
```

Passo 8 — Análise STRIDE

```
stride-analyzer analyze
```

Passo 9 — (Atalho) Pipeline completo

```
stride-analyzer run-all
```

Passo 10 — Testes e lint

```
pytest  
pytest --cov=stride_analyzer --cov-report=term-missing  
ruff check src tests
```

Ao final do passo 8 (ou 9), confira os arquivos em `relatorio/`: `analise_stride_*.json`, `relatorio_stride_*.pdf` e `relatorio_stride_*.docx`.

Alternativa sem download: copie PNGs manualmente para `diagramas_selecionados/` (por exemplo a partir de `minha-seleção-diagramas/`) e execute apenas `stride-analyzer analyze`.

11. Roteiro passo a passo — execução com Docker

11.1 Pré-requisitos

- Docker Desktop 4.x+ (ou Docker Engine + Compose v2)

- Arquivo .env preenchido na raiz do projeto

Passo 1 — Build da imagem

```
docker compose build
```

Passo 2 — Pipeline modular (recomendado)

Cada etapa em um contêiner, encadeada por depends_on:

```
docker compose --profile modular up --abort-on-container-exit
```

Passo 3 — Pipeline monolítico (alternativa)

Equivalente a stride-analyzer run-all:

```
docker compose --profile monolith run --rm pipeline
```

Passo 4 — Executar etapa isolada

```
docker compose --profile modular run --rm download
```

```
docker compose --profile modular run --rm filter
```

```
docker compose --profile modular run --rm analyze
```

Os artefatos ficam disponíveis nas pastas do host mapeadas por bind mount (dataset/, diagramas_selecionados/, relatorio/).

12. Referência de comandos CLI

Comando	Descrição	Exit code
setup	Cria pastas e exibe status	0
download	Baixa dataset Kaggle se necessário	0
filter	Filtra e copia diagramas aprovados	0
filter --skip-clip	Filtra sem carregar o modelo CLIP	0
analyze	STRIDE + gera relatórios	0; 1 se sem PNGs
run-all	download → filter → analyze	primeiro ≠ 0
-v / --verbose	Log DEBUG no stdout	—

Tabela 7 — Comandos do entry point stride-analyzer

13. Testes e qualidade

A suite usa pytest com fixtures em tests/conftest.py (project_root, settings, imagens sintéticas, mock de classificador). Chamadas a Kaggle, OpenAI e CLIP são mockadas — não há dependência de rede nos testes unitários.

Arquivo	Cobertura
test_cli.py	Parser, setup, analyze sem PNGs
test_config.py	Settings e validação de credenciais
test_paths.py	Pastas, listagem de PNGs
test_dataset.py	Download/extração e limpeza XML
test_filter.py	Limite MAX_IMAGENS e rejeição
test_image_analysis.py	Nitidez, alinhamento, CLIP
test_stride.py	Base64 e cliente OpenAI mockado

Arquivo	Cobertura
test_reports.py	Geração JSON/PDF/DOCX

Tabela 8 — Suite de testes

14. Geração de relatórios

A função `salvar_relatorios(resultados, relatorio_dir)` recebe uma lista de dicionários com chaves `arquivo`, `caminho` e `analise` (texto markdown livre do GPT) e produz:

- JSON — `analise_stride_{timestamp}.json` (`ensure_ascii=False`, `indent=2`)
- PDF — ReportLab A4; capa; imagem do diagrama; parágrafos escapados; quebras de página
- DOCX — python-docx; título; imagens; quebras de página entre diagramas

Exemplos de saída de execução real estão em `docs/relatório` final da análise STRIDE/.

15. Solução de problemas

Problema	Solução
Credenciais Kaggle ausentes	Preencha <code>KAGGLE_USERNAME</code> e <code>KAGGLE_KEY</code> no <code>.env</code>
Chave OpenAI ausente	Preencha <code>OPENAI_API_KEY</code> no <code>.env</code>
Nenhum <code>.png</code> encontrado	Execute <code>filter</code> ou copie PNGs para <code>diagramas_selecionados/</code>
Download lento	Normal na 1ª execução; depois usa cache local
Erro / OOM no CLIP	Aumente RAM ou use <code>filter --skip-clip</code> (com ressalvas)
<code>analyze</code> ignora JPG	Converta para PNG ou copie apenas <code>.png</code>

Tabela 9 — Troubleshooting

Documento gerado para o Tech Challenger Fase 5 — Licença MIT. Pacote `stride-analyzer` v1.0.0.
Desenvolvido por Bruno José e Silva - brunojose1977@yahoo.com.br - julho 2026